

Reporte de Actividades 14

- Configuración por ambiente
- Variables de entorno
- User Secrets
- CORS
- Swagger / OpenAPI

En esta parte seguí con los temas de configuración de la API en .NET. La idea principal fue entender cómo una aplicación puede cambiar su configuración dependiendo del ambiente donde se está ejecutando, y también cómo dejar valores sensibles fuera del código.

Configuración por ambiente

La configuración por ambiente sirve para saber si la aplicación está corriendo en Development, Testing, Staging o Production. En el proyecto ya estaba definido el ambiente Development desde el archivo `launchSettings.json` por medio de `ASPNETCORE_ENVIRONMENT`.

Con esto .NET puede leer primero el `appsettings.json` como configuración base y después leer un archivo más específico como `appsettings.Development.json`. Si no existe un archivo del ambiente correspondiente, entonces se queda con la configuración base y otras fuentes como variables de entorno.

En el código también se usó el ambiente para activar Swagger solamente cuando la aplicación está en Development. Esto sirve porque en desarrollo es útil probar la API con Swagger, pero en producción no siempre se quiere exponer esa documentación.

Variables de entorno

Las variables de entorno son configuraciones que viven fuera del proyecto. No están escritas directamente en `Program.cs` ni en `appsettings.json`, sino que se definen en la terminal, en el sistema operativo, en Docker o en el servidor donde se publica la aplicación.

En .NET estas variables se leen por medio de `builder.Configuration`. Por ejemplo, la cadena de conexión AdventureWorks se lee con `GetConnectionString`, y ese valor puede venir de un archivo json, de User Secrets o de una variable de entorno.

Algo que me pareció importante es que cuando una configuración tiene niveles, como `ConnectionStrings:AdventureWorks`, en variables de entorno se escribe con doble guion bajo, por ejemplo `ConnectionStrings__AdventureWorks`.

User Secrets

User Secrets se usa para guardar datos sensibles de manera local durante desarrollo. Esto sirve para no subir al repositorio contraseñas, tokens o cadenas de conexión reales.

En el proyecto se inicializó User Secrets y se guardó ahí la cadena de conexión real de AdventureWorks. Después se quitó del `appsettings.Development.json` para que la contraseña no quedara escrita dentro del proyecto.

También se agregó un `UserSecretsId` en el archivo del proyecto, que es lo que conecta este proyecto con el almacén local de secretos. Esto no sube el secreto, solo deja identificado que el proyecto puede usar User Secrets.

CORS

CORS significa Cross-Origin Resource Sharing. Es una política de seguridad del navegador que controla si un frontend puede hacer peticiones a una API que está en otro origen.

Un origen se forma con el protocolo, dominio y puerto. Por ejemplo, Angular puede estar en `http://localhost:4200` y la API en `http://localhost:5172`. Aunque ambos sean localhost, son orígenes diferentes porque cambia el puerto.

En el proyecto se agregó una política CORS llamada `FrontendCorsPolicy`, que lee los orígenes permitidos desde configuración. En Development se permitieron `http://localhost:4200` y `https://localhost:4200` para que el frontend local pueda llamar al backend.

También entendí que CORS aplica principalmente a navegadores. Si hago una petición desde Postman, curl o desde otro backend, CORS no bloquea la petición. Para esos casos importan otras cosas como que el puerto esté abierto, que la API esté corriendo o que haya autenticación.

Swagger / OpenAPI

Swagger y OpenAPI sirven para documentar y probar la API. OpenAPI es la especificación que describe los endpoints, los métodos HTTP, parámetros, bodies, respuestas y modelos. Swagger es la herramienta visual que permite ver y probar esa documentación desde el navegador.

Para esto se agregó el paquete `Swashbuckle.AspNetCore` y en `Program.cs` se registraron los servicios `AddEndpointsApiExplorer` y `AddSwaggerGen`. Después se activó `UseSwagger` y `UseSwaggerUI` solamente en Development.

Con esto se puede entrar a <http://localhost:5172/swagger> y probar endpoints como obtener productos, crear productos, actualizar o eliminar sin necesidad de usar Postman.

Resumen de lo que se hizo

En resumen, se dejo el proyecto mas preparado para trabajar como una API real. Se separo la configuracion por ambiente, se movio la cadena de conexion real a User Secrets, se agrego CORS para permitir el frontend local y se agrego Swagger para documentar y probar los endpoints.